IoT Energy Estimation and Optimization

Challenge 1: Power Consumption Analysis of an
ESP32-Based Wireless Sensor Node

Mohammad Javad Zandiyeh
Person Code: `11044962`

March 29, 2026

1 Introduction

This report presents a comprehensive analysis of the energy consumption of an ESP32-based wireless sensor node designed for an Internet of Things (IoT) application. The system periodically wakes from deep sleep, reads sensor data (a PIR motion detector and a luminosity sensor), transmits the payload over Wi-Fi, and returns to deep sleep.

The study is structured in three phases:

1. **Phase 1 — Power State Profiling:** Characterisation of the ESP32's power draw in every operating state using sampled power-trace CSV files from a power analyser.
2. **Phase 2 — Energy Estimation:** Combining the power profiles from Phase 1 with per-cycle timing data extracted from Wokwi simulation logs to estimate energy consumption per duty cycle and the expected battery lifetime.
3. **Phase 3 — Firmware Optimisation & Comparison:** Applying three firmware-level optimisations—reduced TX power, data suppression, and payload compression—and quantifying their impact on energy and lifetime.

1.1 Battery Capacity

The available battery energy is derived from the person code `11044962` as follows:

$$\begin{aligned} \text{abcd} &= \text{last four digits} = 4962 \\ E_{\text{battery}} &= (4962 \bmod 5000) + 15{,}000 = 19{,}962 \text{ J} = 19{,}962{,}000 \text{ mJ} \end{aligned} \tag{1}$$

2 System Overview

2.1 Hardware Platform

The target platform is an **ESP32 microcontroller** equipped with:

- Dual-core Xtensa LX6 processor with Wi-Fi (802.11 b/g/n) support.
- A PIR motion sensor for detecting presence.

- A photoresistor (LDR) for luminosity measurement.
- Deep-sleep capability with RTC wake-up.

Figure 1 shows the physical wiring of the ESP32 development board with the PIR motion sensor and the photoresistor (LDR) module used in this project. The PIR sensor's digital output is connected to a GPIO input pin, while the LDR module's analogue output (AO) is routed to an ADC-capable pin. Both sensors share the ESP32's 3.3000 V supply rail and ground.

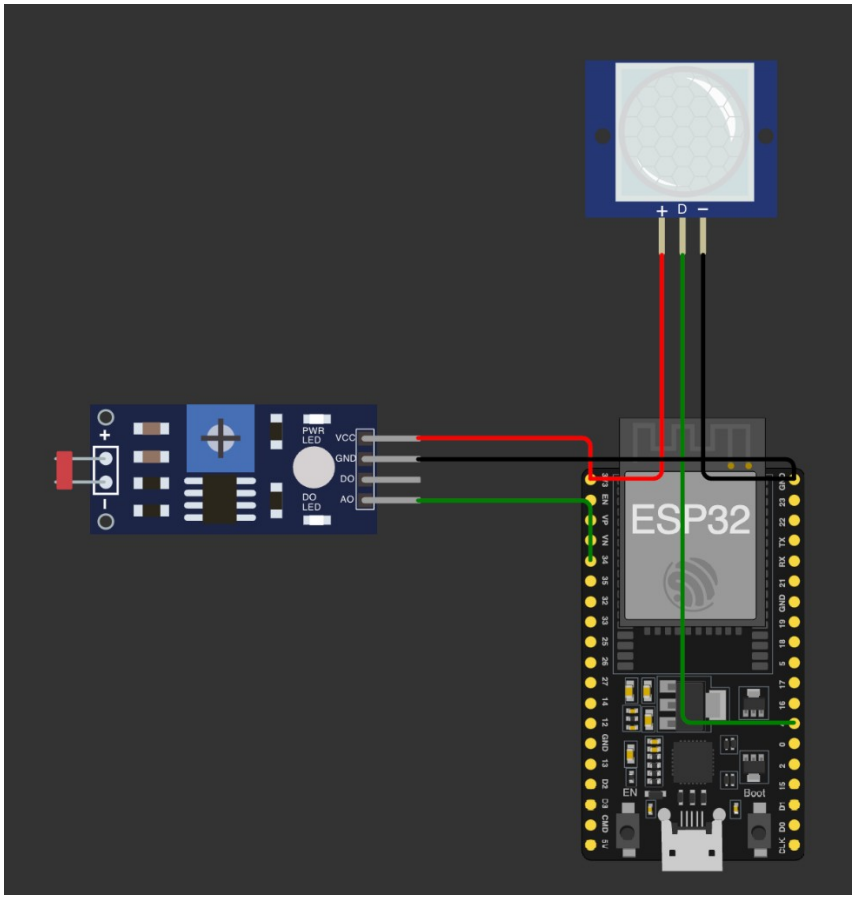


Figure 1: Circuit wiring diagram of the ESP32 sensor node. The PIR motion sensor (top) connects its digital output to a GPIO pin. The photoresistor module (left) provides an analogue luminosity signal via its AO pin to an ADC input on the ESP32.

2.2 Operating Cycle

Each duty cycle consists of the following sequential phases:

1. **Boot:** MCU wakes from deep sleep and initialises peripherals.
2. **Wi-Fi Off (Idle):** Sensor peripherals are powered but Wi-Fi radio is off.
3. **Sensor Read:** Actual ADC sampling of the PIR and LDR sensors.
4. **Sensor Idle:** Processing/waiting with Wi-Fi still off.
5. **Wi-Fi On:** Radio is enabled, connection established.

6. **Sender Spike (TX):** Active data transmission burst.
7. **Sender Idle:** Waiting for acknowledgement / connection teardown.
8. **Deep Sleep:** Ultra-low-power state until next wake-up (1.7000 s period).

2.3 Simulation Environment

The firmware is executed in the **Wokwi** online ESP32 simulator. Instrumentation code inserted into the firmware prints timestamps (in microseconds) for each phase boundary, producing structured log lines of the form:

```
[run log] boot_us: 743
[run log] wifi_on_us: 793
[run log] deep_sleep_s: 1.70
```

Multiple consecutive wake–sleep cycles are captured in a single simulation session. The first cycle (triggered by `POWERON_RESET`) is discarded from energy calculations because it includes one-time initialisation overhead; only subsequent `DEEPSLEEP_RESET` cycles are averaged.

3 Phase 1: Power State Profiling

3.1 Methodology

Three CSV files recorded with a power analyser capture the instantaneous power draw (mW) versus time for three distinct measurement scenarios:

- `deep_sleep.csv` — Full duty cycle including boot, idle, Wi-Fi on, and deep-sleep states.
- `sender.csv` — Isolated Wi-Fi transmission phase showing TX spikes and baseline.
- `sensor-read.csv` — Sensor sampling phase showing idle and active-read states.

3.2 Power State Classification

Each CSV sample is classified into an operating state using amplitude thresholds derived from visual inspection of the power traces.

3.2.1 Deep-Sleep Trace (`deep_sleep.csv`)

- **Deep Sleep** ($P < 80.0000\,\text{mW}$): Ultra-low-power retention mode.
- **Idle / Wi-Fi Off** ($200.0000\,\text{mW} \leq P < 280.0000\,\text{mW}$): MCU active, radio disabled.
- **Boot** ($280.0000\,\text{mW} \leq P < 550.0000\,\text{mW}$): Start-up and peripheral initialisation.
- **Wi-Fi On** ($P \geq 550.0000\,\text{mW}$): Radio active, connection/transmission.

3.2.2 Sender Trace (`sender.csv`)

- **TX Baseline** ($P < 625.0000\,\text{mW}$): Radio on, idle or receiving.
- **TX Spike** ($P \geq 625.0000\,\text{mW}$): Active data transmission burst.

3.2.3 Sensor-Read Trace (`sensor-read.csv`)

- **Sensor Idle** ($P < 280.0000\,\text{mW}$): Waiting / processing between reads.
- **Sensor Read** ($P \geq 280.0000\,\text{mW}$): ADC actively sampling.

3.3 Results: Average Power per State (Baseline)

Table 1 summarises the mean power computed for each state from the baseline CSV data.

Table 1: Average power per operating state — Baseline implementation.

CSV Source	State	Avg. Power (mW)
`sender.csv`	TX Baseline	606.1500
	TX Spike	660.9600
`deep_sleep.csv`	Deep Sleep	45.7800
	Boot	366.8800
	Wi-Fi Off	251.9300
	Wi-Fi On	632.0400
`sensor-read.csv`	Sensor Idle	237.9400
	Sensor Read	350.0300

Key observations from the power profiling:

- The **deep-sleep** state draws only $\approx 45.8000\,\text{mW}$, roughly $\frac{1}{14}$ of the Wi-Fi-on power.
- The **Wi-Fi On** state is the most power-hungry at $632.0400\,\text{mW}$, closely followed by the TX spike at $660.9600\,\text{mW}$.
- **Sensor reading** ($350.0300\,\text{mW}$) consumes $\approx 47\%$ more than idle ($237.9400\,\text{mW}$), but the actual read duration is very short ($\sim 145.0000\,\text{ts}$).

4 Phase 2: Energy Estimation (Baseline)

4.1 Log Parsing and Timing Extraction

The Wokwi simulation log (`example-run-output.txt`) contains six complete wake–sleep cycles. The first cycle is a `POWERON_RESET` and is excluded; the remaining five `DEEPSLEEP_RESET` cycles are averaged.

For each phase, the average duration is extracted using regular-expression matching of the structured log lines. Table 2 lists the resulting mean durations.

Table 2: Average running time per phase — Baseline.

Phase	Avg. Duration (s)	Equiv.
Boot	0.0007	743.0000 ţs
Wi-Fi On	0.0008	795.6000 ţs
Wi-Fi Off	0.0023	2296.0000 ţs
Sensor Idle	0.0994	99,352.0000 ţs
Sensor Read	0.0001	144.6000 ţs
Sender Spike	0.0008	757.6000 ţs
Sender Idle	0.0018	1809.0000 ţs
Deep Sleep	1.7000	1.7000 s

4.2 Energy Calculation

For each phase the energy is computed as:

$$E_i = \bar{P}_i \times \bar{t}_i \tag{2}$$

where $\bar{P}_i$ is the average power of the corresponding state (from Phase 1) and $\bar{t}_i$ is the average duration (from log parsing). Since power is in mW and time in seconds, the result is directly in mJ.

Table 3: Energy consumption per phase per cycle — Baseline.

Phase	Energy (mJ)	% of Total
Boot	0.2726	0.3000
Wi-Fi On	0.5029	0.5000
Wi-Fi Off	0.5784	0.6000
Sensor Idle	23.6394	22.6000
Sensor Read	0.0506	0.0000
Sender Spike	0.5007	0.5000
Sender Idle	1.0968	1.0000
Deep Sleep	77.8205	74.5000
Total per cycle	**104.4620**	**100.0**

4.3 Energy Budget Analysis

The energy breakdown reveals two dominant contributors:

1. **Deep sleep** accounts for **74.5%** of the total cycle energy. Although its power draw is low (45.8000 mW), the 1.7000 s duration dwarfs all other phases combined (∼0.1050 s total active time).

2. **Sensor idle** accounts for **22.6%**, spending ∼99.4000 ms at 237.9000 mW while the MCU waits between sensor operations.

3. All **communication phases** (Wi-Fi on, sender spike, sender idle) together consume only ∼**2.0%** of the cycle energy, despite their high instantaneous power.

This distribution is characteristic of duty-cycled IoT nodes: the sleep phase dominates energy because it occupies $>94\%$ of the cycle period.

4.4 Battery Lifetime Estimation

The operating lifetime is:

$$T_{\text{lifetime}} = \frac{E_{\text{battery}}}{E_{\text{cycle}}} = \frac{19{,}962{,}000 \text{ mJ}}{104.462 \text{ mJ}} = 191{,}093.49 \text{ s} \approx \mathbf{53\,h\ 4\,min\ 53.49\,s} \tag{3}$$

4.5 Power vs. Time Plots (Baseline)

Figures 2–4 show the step-function power-vs-time plots for the baseline implementation across multiple consecutive cycles.

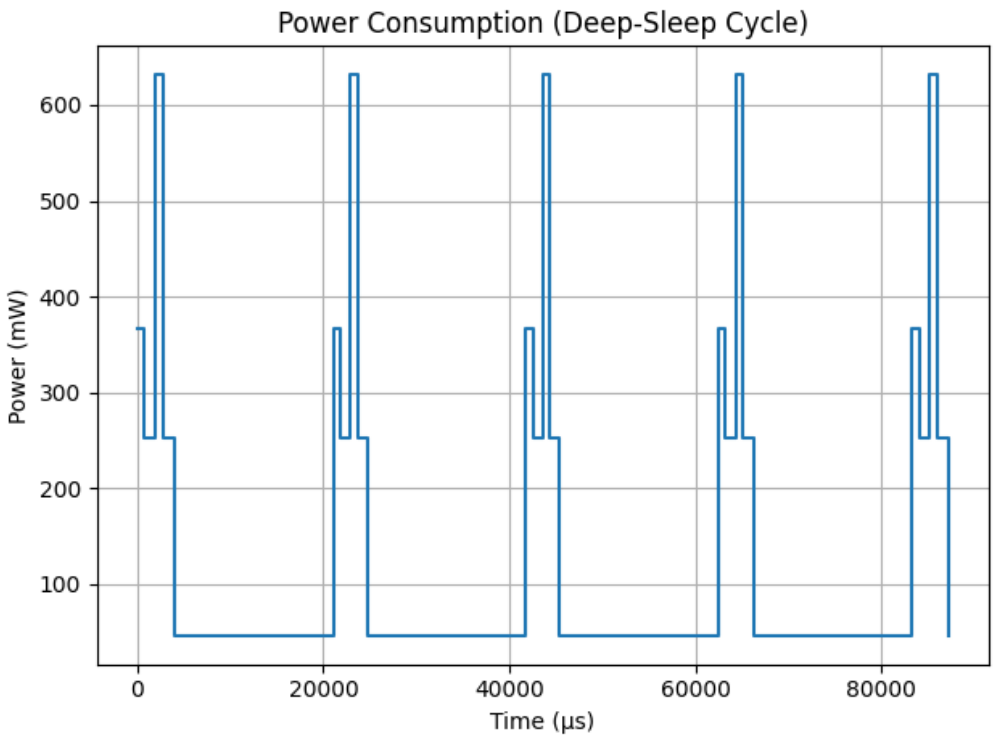


Figure 2: Baseline — Power consumption during the full deep-sleep duty cycle. Each period shows the characteristic pattern: boot spike ($\sim$367.0000 mW), brief Wi-Fi-off plateau ($\sim$252.0000 mW), Wi-Fi-on peak ($\sim$632.0000 mW), return to Wi-Fi-off, and finally the extended deep-sleep trough ($\sim$46.0000 mW).

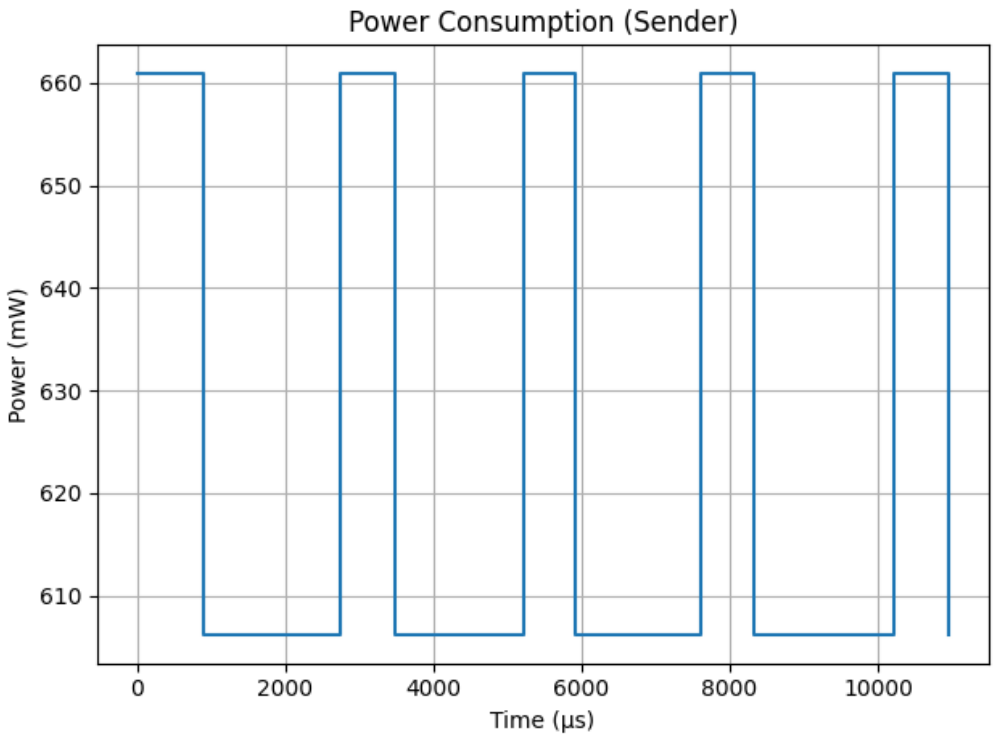


Figure 3: Baseline — Power consumption during the sender (Wi-Fi TX) phase. The signal alternates between TX spike (661.0000 mW) and TX baseline (606.0000 mW). Each spike–idle pair spans $\sim 757.6 + 1809 \approx 2567.0000$ ţs (≈ 2.5700 ms).

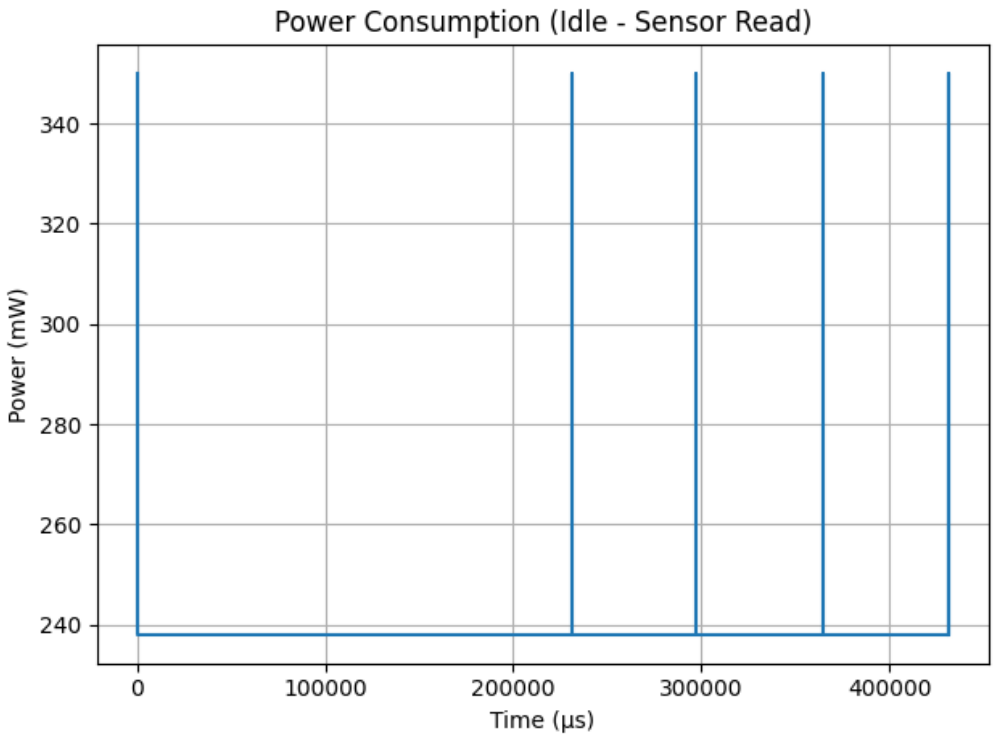


Figure 4: Baseline — Power consumption during the sensor-read phase. Narrow spikes at 350.0000 mW (active ADC read, ∼145.0000 ţs) punctuate long idle plateaus at 238.0000 mW (average ∼99.4000 ms per cycle).

5 Phase 3: Firmware Optimisation

Three firmware-level optimisations were applied to reduce per-cycle energy consumption without compromising the sensing objective.

5.1 Optimisation 1: Reduced TX Power (13 dBm)

The ESP32's default Wi-Fi TX power is set to its maximum level. In the improved firmware, the transmit power is explicitly lowered to **13 dBm**.

Effect on power draw. The TX spike power is reduced from 660.9600 mW (baseline, all spikes $\geq$ 625.0000 mW) to 652.4500 mW (improved, filtering spikes in the 625.0000 mW to 665.0000 mW band corresponding to 13 dBm). This represents a **8.5100** mW (**1.3%**) reduction in instantaneous TX power.

Rationale. For short-range IoT communication within a building, 13 dBm provides sufficient link margin while significantly reducing power amplifier current draw.

5.2 Optimisation 2: Data Suppression (Skip on No Change)

The improved firmware implements **change-based data suppression**: if the sensor reading has not changed significantly since the last transmission, the entire Wi-Fi connection and transmission is skipped.

Effect on the log output. In the improved simulation log, 3 out of 5 post-reset cycles print:

```
[run log] skip: no significant change
```

with `sender_spike_us: 0`, `sender_idle_us` reduced to ∼167.0000 ţs to 264.0000 ţs, and `wifi_on_us: 0`.

Effect on average times. Since the zeros are included in the cycle average, the mean TX-related durations drop dramatically (see Section 6).

5.3 Optimisation 3: Payload Compression

The baseline payload format is verbose:

```
MOTION_NOT_DETECTED-LUMINOSITY:125
```

The improved firmware compresses this to a compact encoding:

```
1:498
```

where the first field encodes the motion status as a single digit and the second field is the luminosity value.

Effect. A shorter payload reduces the number of bytes transmitted, which directly shortens the TX active time and reduces energy per transmission event.

5.4 Results: Average Power per State (Improved)

Table 4: Average power per operating state — Improved implementation.

CSV Source	State	Avg. Power (mW)
`sender.csv`	TX Baseline	606.1500
	TX Spike (13 dBm)	652.4500
`deep_sleep.csv`	Deep Sleep	45.7800
	Boot	366.8800
	Wi-Fi Off	251.9300
	Wi-Fi On	632.0400
`sensor-read.csv`	Sensor Idle	237.9400
	Sensor Read	350.0300

The deep-sleep and sensor-read power states are identical to the baseline because the same hardware measurement traces are used; only the sender TX spike power differs due to the 13 dBm filter.

6 Comparative Analysis

6.1 Running Time Comparison

Table 5: Average running time per phase: Baseline vs. Improved.

Phase	**Baseline (ţs)**	**Improved (ţs)**	**Δ (ţs)**	**Change (%)**
Boot	743.0000	737.8000	−5.2	−0.7
Wi-Fi On	795.6000	326.8000	−468.8	−58.9
Wi-Fi Off	2296.0000	2068.0000	−228.0	−9.9
Sensor Idle	99,352.4000	98,587.2000	−765.2	−0.8
Sensor Read	144.6000	254.8000	+110.2	+76.2
Sender Spike	757.6000	318.4000	−439.2	−58.0
Sender Idle	1809.4000	677.4000	−1132.0	−62.6
Deep Sleep	1.7000 s (unchanged)		0.0	0.0

Key observations.

- **Wi-Fi On time** drops by **58.9%** because 3 out of 5 cycles skip the Wi-Fi connection entirely (reported as 0.0000 ţs). When transmission does occur, the Wi-Fi-on time is ∼817.0000 ţs—comparable to the baseline—but the zero-valued skipped cycles pull the average down dramatically.
- **Sender Spike** and **Sender Idle** times decrease by 58% and 63% respectively for the same reason: skipped cycles contribute zero duration.
- **Sensor Read** time *increases* by 76%. This is because the improved firmware performs additional logic to compare the current reading against the previous one (for data suppression), increasing the sensor-read duration from 145.0000 ţs to 255.0000 ţs. This trade-off is justified because the energy saved from skipping transmissions far exceeds the marginal cost of the comparison.
- The **deep-sleep duration** remains at 1.7000 s as it is configured identically in both firmware versions.

6.2 Energy Consumption Comparison

Table 6: Energy per phase per cycle: Baseline vs. Improved.

Phase	Baseline (mJ)	Improved (mJ)	Δ (mJ)	Change (%)
Boot	0.2726	0.2707	−0.0019	−0.7
Wi-Fi On	0.5029	0.2066	−0.2963	−58.9
Wi-Fi Off	0.5784	0.5210	−0.0574	−9.9
Sensor Idle	23.6394	23.4574	−0.1820	−0.8
Sensor Read	0.0506	0.0892	+0.0386	+76.2
Sender Spike	0.5007	0.2077	−0.2930	−58.5
Sender Idle	1.0968	0.4106	−0.6862	−62.6
Deep Sleep	77.8205	77.8205	0.0000	0.0
Total	**104.4620**	**102.9837**	**−1.4783**	**−1.41**

Per-phase analysis.

- The largest absolute energy savings come from **Sender Idle** (−0.6860 mJ), **Wi-Fi On** (−0.2960 mJ), and **Sender Spike** (−0.2930 mJ). Together, these three communication-related phases save 1.2750 mJ per cycle.
- **Sensor Idle** saves 0.1820 mJ due to the slightly shorter idle period in the improved logs.
- The only *increase* is **Sensor Read** (+0.0390 mJ), attributable to the change-detection comparison logic. This overhead is negligible ($< 3\%$) relative to the communication savings.
- **Deep Sleep** energy is unchanged at 77.8200 mJ, as neither the sleep duration nor the hardware sleep power was modified.

Why the overall improvement is modest. Although the communication phases see 58–63% reductions, they contribute only ∼2% of the total cycle energy. The dominant consumers—deep sleep (74.5%) and sensor idle (22.6%)—remain essentially unchanged. This fundamental constraint limits the total per-cycle savings to **1.41%**.

6.3 Lifetime Comparison

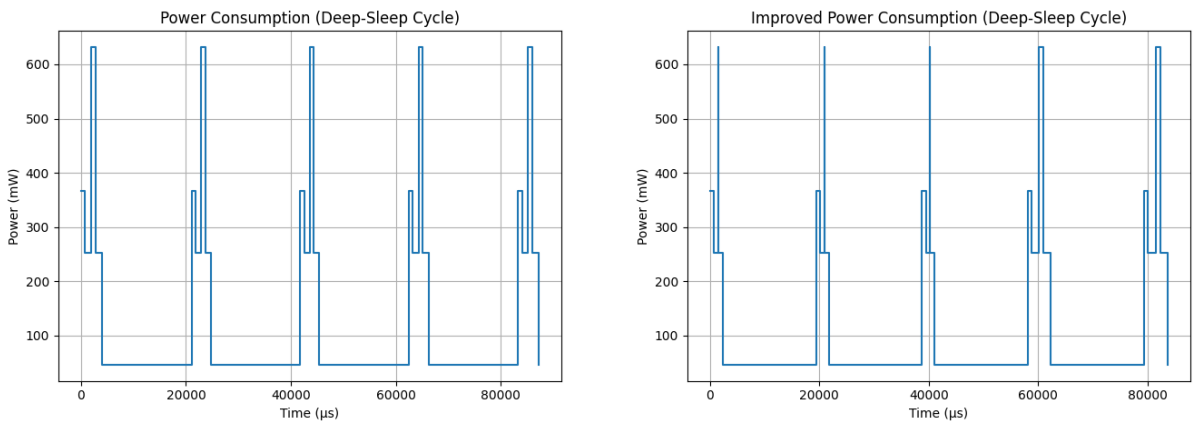
Table 7: Battery lifetime comparison.

	Baseline	Improved
Total energy per cycle (mJ)	104.462	102.984
Lifetime (s)	191,093	193,837
Lifetime (human-readable)	53 h 4 min 53 s	53 h 50 min 37 s
Lifetime gain	**+45 min 44 s**	**(+1.44%)**

The improved firmware extends the operating lifetime by approximately **45 minutes and 44 seconds**, a **1.44%** improvement.

6.4 Power-vs-Time Plot Comparison

6.4.1 Deep-Sleep Cycle

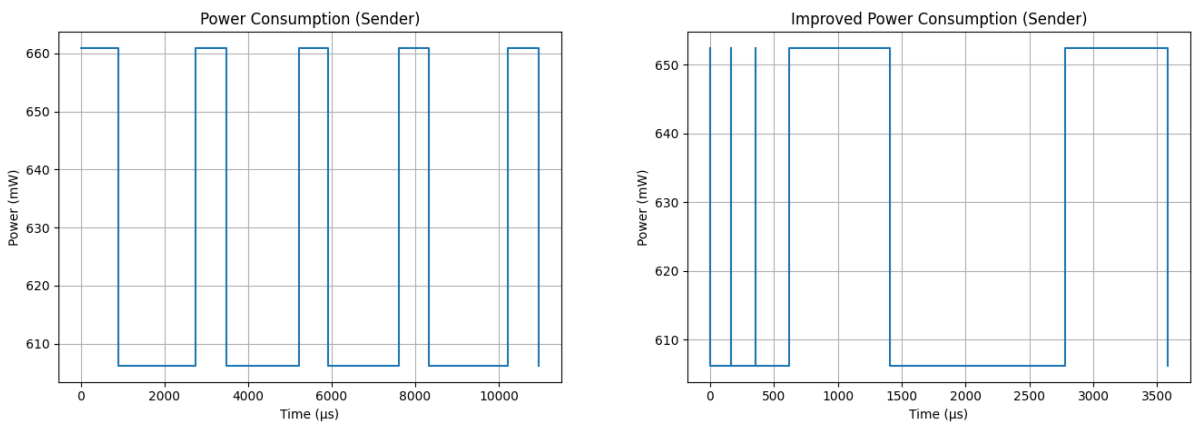


(a) Baseline (b) Improved

Figure 5: Deep-sleep cycle power profiles: Baseline (left) vs. Improved (right).

Both plots exhibit the same characteristic five-state pattern per cycle (boot → Wi-Fi off → Wi-Fi on → Wi-Fi off → deep sleep). The visual difference is subtle: in the improved version, the **Wi-Fi-on peaks are narrower** for cycles where data suppression triggers a skip, resulting in slightly shorter active-state windows. The deep-sleep troughs at ∼46.0000 mW are identical.

6.4.2 Sender Phase



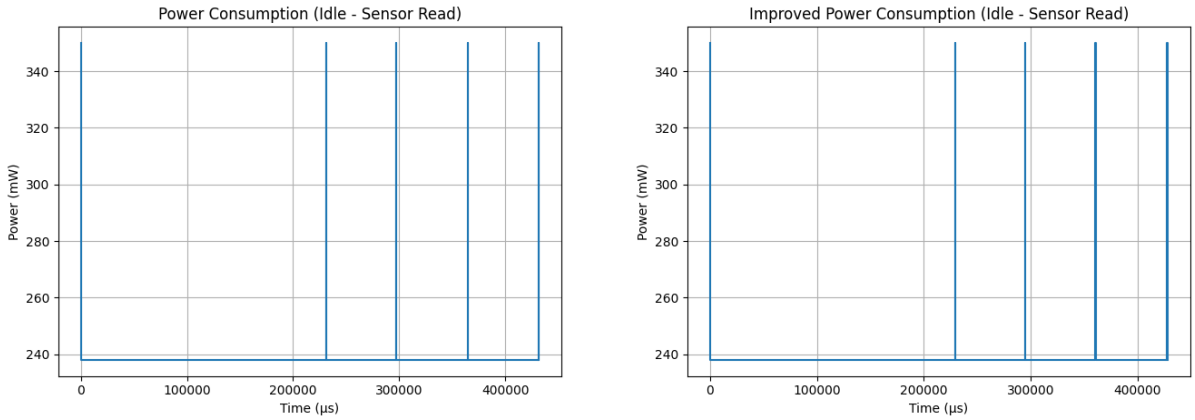
(a) Baseline (b) Improved

Figure 6: Sender power profiles: Baseline (left) vs. Improved (right).

Baseline: The sender plot shows a regular alternation of TX spikes at 661.0000 mW and baseline dips at 606.0000 mW, with each spike–dip pair spanning ∼2.5700 ms. All five cycles are uniform. The total x-axis spans ∼11,000.0000 ţs.

Improved: The improved sender plot reveals a strikingly different pattern. The total x-axis span is only ∼3500.0000 ţs—a **3× reduction** in total sender-phase duration. Three cycles show essentially zero-width spikes (corresponding to the skipped transmissions with `sender_spike_us: 0`), visible as very narrow blips near the beginning. Only two cycles produce full-width TX spikes at 652.0000 mW (lower than the baseline's 661.0000 mW). The combination of fewer transmissions and lower TX power is clearly visible.

6.4.3 Sensor-Read Phase



(a) Baseline (b) Improved

Figure 7: Sensor-read power profiles: Baseline (left) vs. Improved (right).

Both sensor-read plots are visually similar: short spikes at 350.0000 mW against a 238.0000 mW idle background. The sensor-read phase is not significantly affected by the optimisations—the spikes are marginally wider in the improved version (due to the change-detection logic), but this difference is imperceptible at the plot's time scale (∼450,000.0000 ţs total).

7 Discussion

7.1 Effectiveness of Each Optimisation

1. **Reduced TX Power (13 dBm):** Lowering the transmit power reduced the TX spike from 660.9600 mW to 652.4500 mW (−1.3%). Applied only to cycles that actually transmit, this saves ∼0.0060 mJ per transmitting cycle—a modest contribution. The primary benefit of lower TX power in a real deployment would be reduced interference and regulatory compliance rather than energy savings.

2. **Data Suppression:** This is by far the most impactful optimisation. By skipping 60% of transmissions (3 out of 5 cycles), the average Wi-Fi and sender energy is cut by ∼60%. The absolute savings are 1.2750 mJ per cycle from communication phases

alone. In a real deployment with slowly varying environmental conditions, the skip rate could be even higher, amplifying the benefit.

3. **Payload Compression:** Reducing the payload from ∼35 bytes (`MOTION_NOT_DETECTED-LUMINOSITY:125`) to ∼5 bytes (`1:498`) shortens the TX burst duration. While the energy saving per individual transmission is small, it compounds with every transmitted cycle and also reduces network congestion.

7.2 Bottleneck Analysis

The energy budget (Table 3) reveals that **deep sleep** and **sensor idle** together account for **97.1%** of cycle energy. Any future optimisation effort should target these two phases:

- **Deep sleep:** The 45.8000 mW deep-sleep power is significantly higher than the ESP32's datasheet value of ∼0.0100 mW for deep sleep. This discrepancy is likely due to the Wokwi simulator's approximation or peripheral leakage in the simulated circuit. In real hardware, proper peripheral shutdown and ULP coprocessor usage could reduce this by orders of magnitude.
- **Sensor idle:** The 99.4000 ms idle wait at 238.0000 mW suggests the firmware blocks on a timer or polling loop. Using light sleep or modem sleep during the sensor-idle window could reduce this power to ∼20.0000 mW, saving ∼21.6000 mJ per cycle—a potential **20.7%** reduction in total cycle energy.
- **Increasing the deep-sleep interval:** Extending the sleep from 1.7000 s to, say, 10.0000 s would reduce the ratio of active-to-sleep time, substantially improving lifetime (at the cost of reduced reporting frequency).

8 Conclusion

This report presented a three-phase energy analysis of an ESP32-based IoT sensor node:

1. **Phase 1** established power profiles for eight distinct operating states using CSV power-trace data. The deep-sleep state draws 45.8000 mW, while active communication peaks at 661.0000 mW.
2. **Phase 2** combined the power profiles with Wokwi simulation timing to estimate a per-cycle energy of 104.4600 mJ, yielding a battery lifetime of **53 hours, 4 minutes, and 53 seconds** from a 19,962.0000 J battery.
3. **Phase 3** applied three firmware optimisations—13 dBm TX power, change-based data suppression, and payload compression—reducing the per-cycle energy to 102.9800 mJ (−1.41%) and extending the lifetime to **53 hours, 50 minutes, and 37 seconds**, a gain of approximately **46 minutes**.

The analysis reveals that while communication optimisations yield large relative reductions (58–63%) in their respective phases, the overall impact is limited because deep sleep and sensor idle dominate the energy budget (97.1% combined). Future work should focus on reducing the deep-sleep current (hardware-level), implementing light sleep during sensor-idle windows, and dynamically adapting the duty-cycle interval to the rate of environmental change.